

Deliverable 1: API Low-Level Design (LLD)

1. REST API Endpoints

Order Service (<http://localhost:8080/api/orders>)

POST / (Place Order)

- **Description:** Creates a new order, saves it to the database with status PLACED, and publishes an order.created event to ActiveMQ.

- **Request Payload (JSON):**

```
json
{
  "customerName": "John Doe",
  "item": "Hyderabadi Chicken Biryani",
  "amount": 240.00
}
```

- **Response Payload (JSON - 200 OK):**

```
json
{
  "id": 1,
  "customerName": "John Doe",
  "item": "Hyderabadi Chicken Biryani",
  "amount": 240.00,
  "status": "PLACED",
  "createdAt": "2026-06-16T10:00:00Z"
}
```

GET / (Get All Orders):

- **Response:** Array of Order objects.

GET /{id} (Get Order by ID)

- **Response:** Order object or 404 Not Found.

Payment Service (<http://localhost:8081/api/payments>)

POST / (Process Payment)

- **Request Payload (JSON):**

```
json
{
  "orderId": 1,
  "amount": 240.00
}
```

Response Payload (JSON - 200 OK):

```
json
{
  "id": 1,
  "orderId": 1,
  "amount": 240.00,
  "status": "SUCCESS",
  "paymentDate": "2026-06-16T10:00:05Z"
}
```

Kitchen Service (<http://localhost:8082/api/kitchen-tickets>)

POST / (Create Kitchen Ticket)

- Request Payload (JSON):

```
json
{
  "orderId": 1,
  "item": "Hyderabadi Chicken Biryani"
}
```

Response Payload (JSON - 200 OK):

```
json
{
  "id": 1,
  "orderId": 1,
  "item": "Hyderabadi Chicken Biryani",
  "status": "READY",
  "ticketCreated": "2026-06-16T10:01:00Z"
}
```

Delivery Service (<http://localhost:8083/api/deliveries>)

POST / (Assign Delivery)

- Request Payload (JSON):

```
json
{
  "orderId": 1
}
```

- Response Payload (JSON - 200 OK):

```
json
{
  "id": 1,
  "orderId": 1,
  "driverName": "Driver-1",
  "status": "DELIVERED",
  "deliveryDate": "2026-06-16T10:30:00Z"
}
```

2. ActiveMQ Queue Message Formats

Queue Name: `order.created`

- **Producer:** `OrderController`
- **Consumer:** `ActiveMQListener` (Order Service)
- **Message Format:** Java `Map<String, Object>` serialized to JMS `MapMessage/JSON`.

json

```
{
  "orderId": 1,
  "customerName": "John Doe",
  "item": "Hyderabadi Chicken Biryani",
  "amount": 240.00
}
```

3. Error Handling and Edge Cases

Delegate Fallbacks (Camunda Workflow)

1. Payment Failure:

- If the Payment service returns a status other than `SUCCESS` or is unreachable, the Camunda workflow delegate (`PaymentServiceTask`) logs an error and updates the order status to `PAYMENT_FAILED`. The process variable `paymentSuccess` is set to `false`, allowing the BPMN diagram to route to a cancellation path if configured.

2. Kitchen Service Unreachable:

- If the Kitchen Service throws an exception, `KitchenServiceTask` logs the error but maintains robustness. In a production scenario, this would trigger an incident or a retry mechanism.

3. Delivery Service Fail-safe:

- If the Delivery Service API call fails (e.g., service is down), `DeliveryServiceTask` catches the exception and executes a fail-safe fallback: it marks the local order as `DELIVERED` to prevent workflow deadlocks in this demonstration environment.

General API Error Handling

- Controllers use Spring Boot's default error handling for missing payloads (400 Bad Request) and unhandled exceptions (500 Internal Server Error).
- Cross-Origin Resource Sharing (CORS) is enabled (`@CrossOrigin(origins = "*")`) to prevent browser preflight errors from the React frontend.

Deliverable 2: Database Design

Database Schema Details

The system uses a distributed database architecture, with each microservice owning its corresponding database, ensuring loose coupling and adherence to microservices best practices.

1. `order_db.orders` (Order Service)

Tracks the overarching state of an order initiated by the customer.

Column	Type	Constraints	Description
<code>id</code>	<code>BIGINT</code>	<code>PRIMARY KEY</code> , <code>AUTO_INCREMENT</code>	Unique identifier for the order.
<code>customer_name</code>	<code>VARCHAR(255)</code>	<code>NOT NULL</code>	Name of the customer placing the order.
<code>item</code>	<code>VARCHAR(255)</code>	<code>NOT NULL</code>	Summary of items ordered.
<code>amount</code>	<code>DOUBLE</code>	<code>NOT NULL</code>	Total cost of the order.
<code>status</code>	<code>VARCHAR(50)</code>	<code>NOT NULL</code>	Order lifecycle state (e.g., <code>PLACED</code> , <code>OUT_FOR_DELIVERY</code>).
<code>created_at</code>	<code>TIMESTAMP</code>	<code>DEFAULT CURRENT_TIMESTAMP</code>	Timestamp of order creation.

2. `payment_db.payments` (Payment Service)

Records payment transactions related to orders.

Column	Type	Constraints	Description
<code>id</code>	<code>BIGINT</code>	<code>PRIMARY KEY</code> , <code>AUTO_INCREMENT</code>	Unique identifier for the payment.
<code>order_id</code>	<code>BIGINT</code>	<code>NOT NULL</code>	Logical Foreign Key to <code>order_db.orders.id</code> .
<code>amount</code>	<code>DOUBLE</code>	<code>NOT NULL</code>	Amount processed.
<code>status</code>	<code>VARCHAR(50)</code>	<code>NOT NULL</code>	Transaction state (e.g., <code>SUCCESS</code>).
<code>payment_date</code>	<code>TIMESTAMP</code>	<code>DEFAULT CURRENT_TIMESTAMP</code>	Timestamp of the transaction.

3. `kitchen_db.kitchen_tickets` (Kitchen Service)

Tracks food preparation tickets.

Column	Type	Constraints	Description
<code>id</code>	<code>BIGINT</code>	<code>PRIMARY KEY</code> , <code>AUTO_INCREMENT</code>	Unique identifier for the ticket.
<code>order_id</code>	<code>BIGINT</code>	<code>NOT NULL</code>	Logical Foreign Key to <code>order_db.orders.id</code> .
<code>item</code>	<code>VARCHAR(255)</code>	<code>NOT NULL</code>	Items to be prepared.
<code>status</code>	<code>VARCHAR(50)</code>	<code>NOT NULL</code>	Preparation state (e.g., <code>READY</code>).
<code>ticket_created</code>	<code>TIMESTAMP</code>	<code>DEFAULT CURRENT_TIMESTAMP</code>	Timestamp of ticket creation.

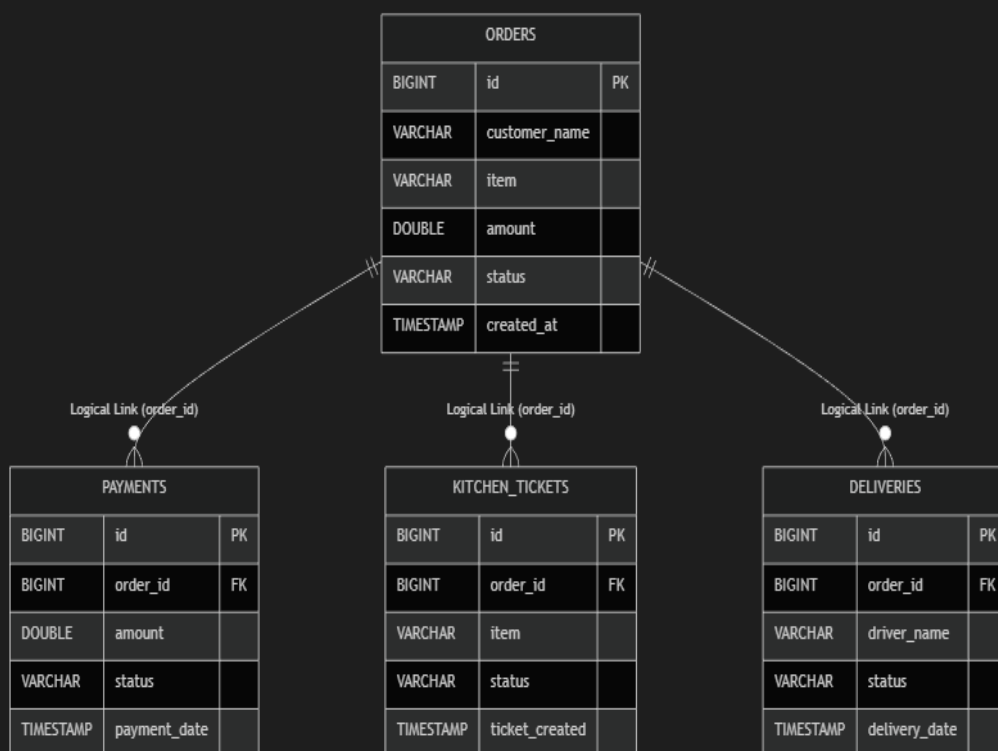
4. `delivery_db.deliveries` (Delivery Service)

Manages delivery partner assignments and statuses.

Column	Type	Constraints	Description
<code>id</code>	<code>BIGINT</code>	<code>PRIMARY KEY</code> , <code>AUTO_INCREMENT</code>	Unique identifier for the delivery.
<code>order_id</code>	<code>BIGINT</code>	<code>NOT NULL</code>	Logical Foreign Key to <code>order_db.orders.id</code> .
<code>driver_name</code>	<code>VARCHAR(255)</code>		Assigned delivery driver.
<code>status</code>	<code>VARCHAR(50)</code>	<code>NOT NULL</code>	Delivery state (e.g., <code>DELIVERED</code>).
<code>delivery_date</code>	<code>TIMESTAMP</code>	<code>DEFAULT CURRENT_TIMESTAMP</code>	Timestamp of assignment/delivery.

Entity Relationship Diagram

Because this is a microservices architecture, the tables reside in separate physical databases (`order_db`, `payment_db`, `kitchen_db`, `delivery_db`). The relationships below are **Logical Foreign Keys**, meaning they are enforced via application logic (by passing `order_id` over REST APIs) rather than strict RDBMS constraints.

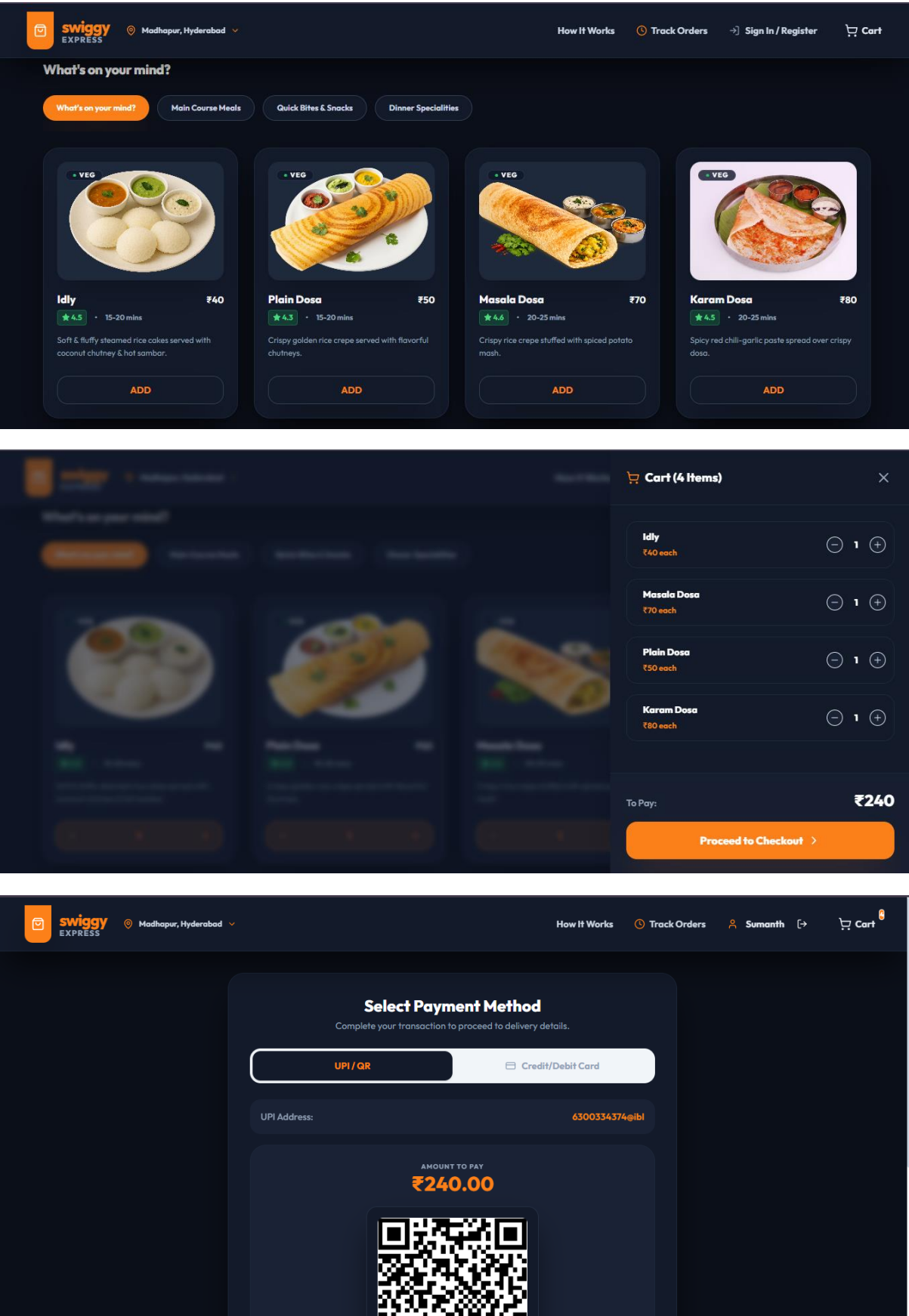


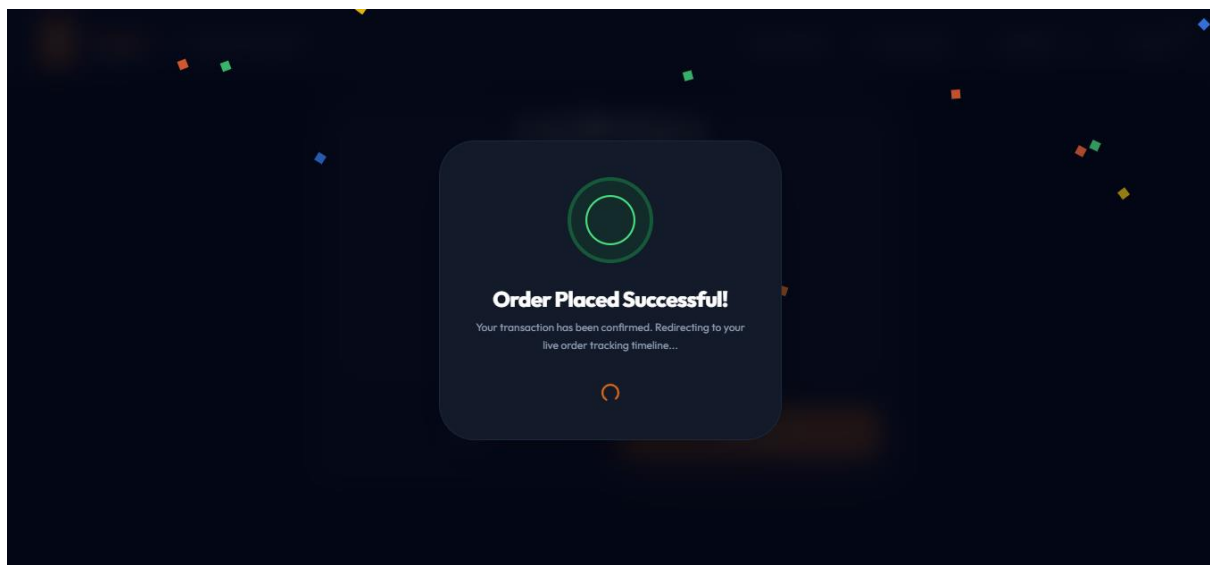
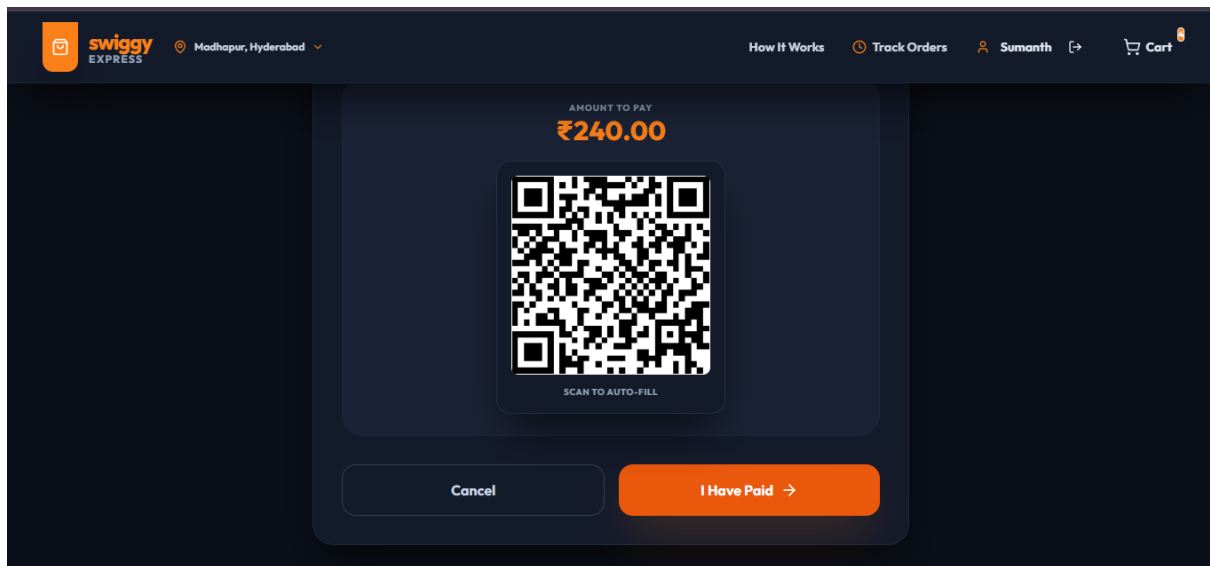
100%

Copy

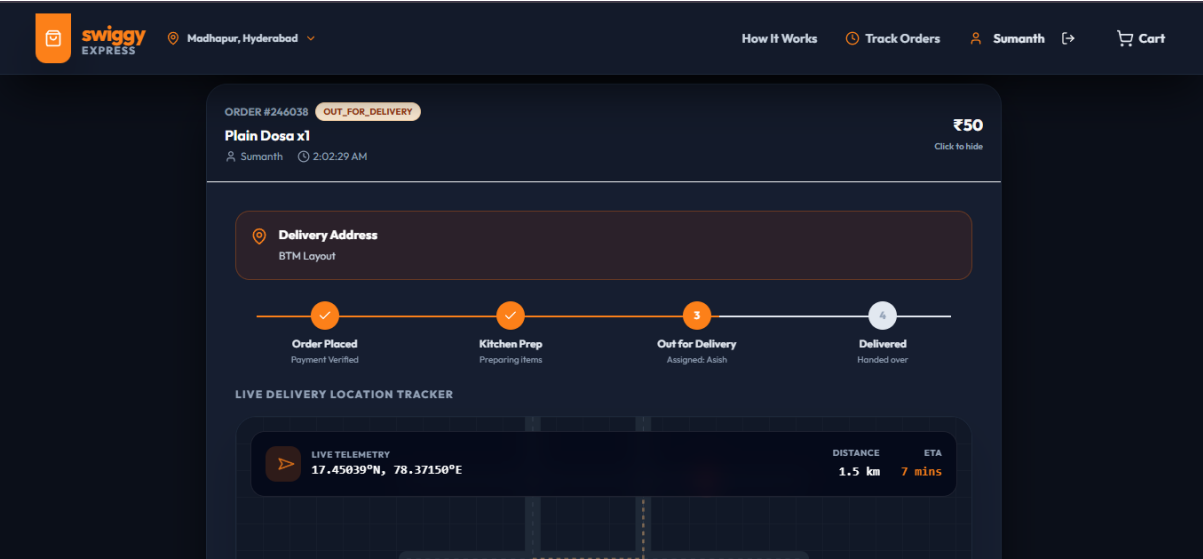
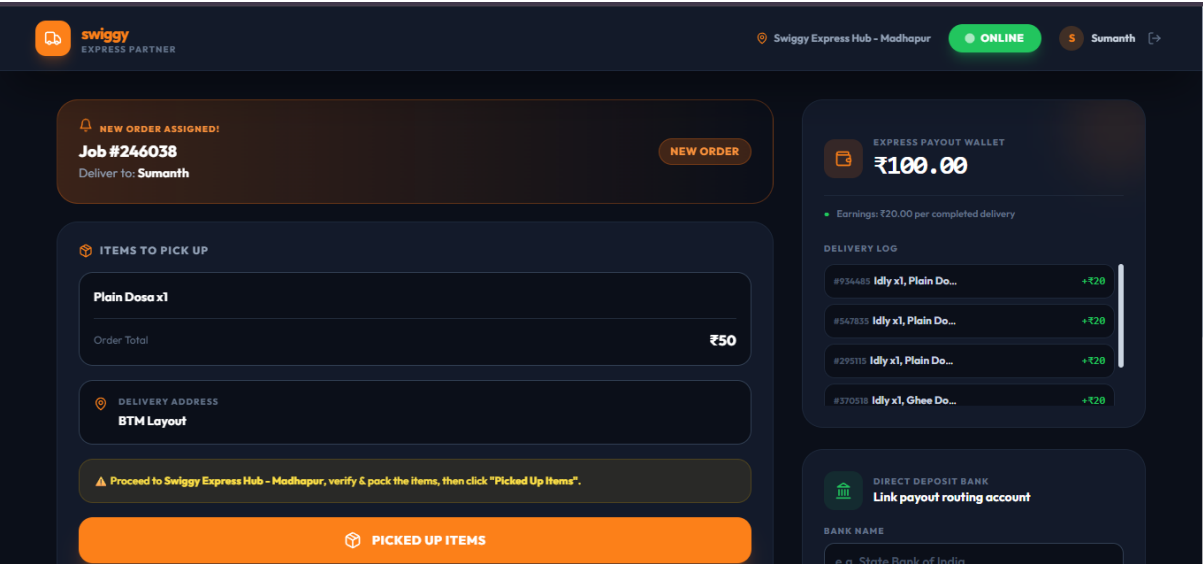
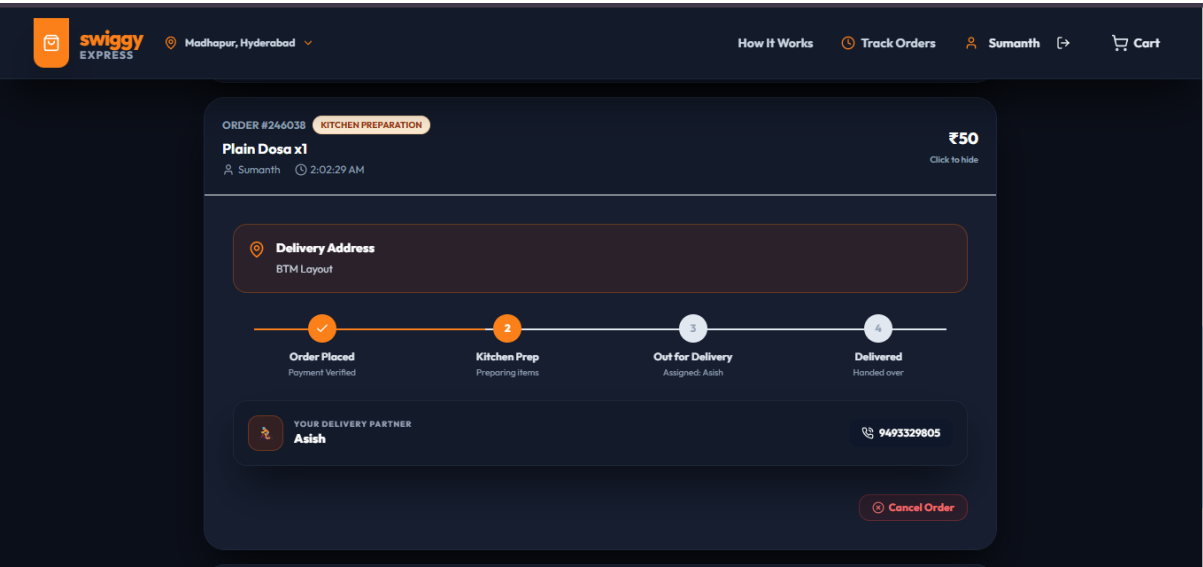
Deliverable 3: Frontend Screenshots

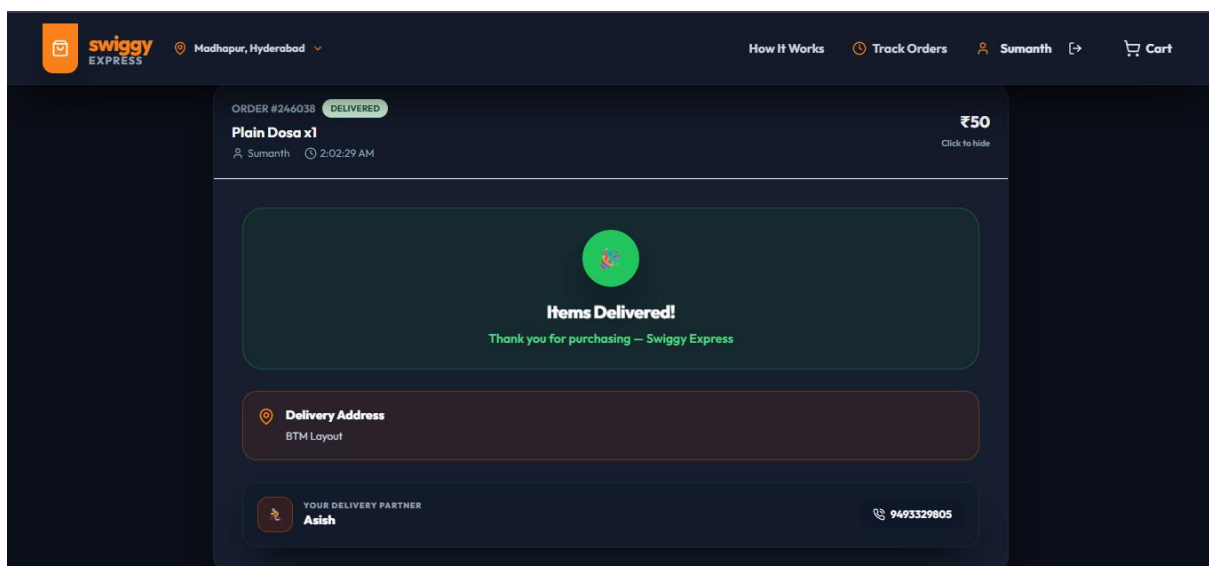
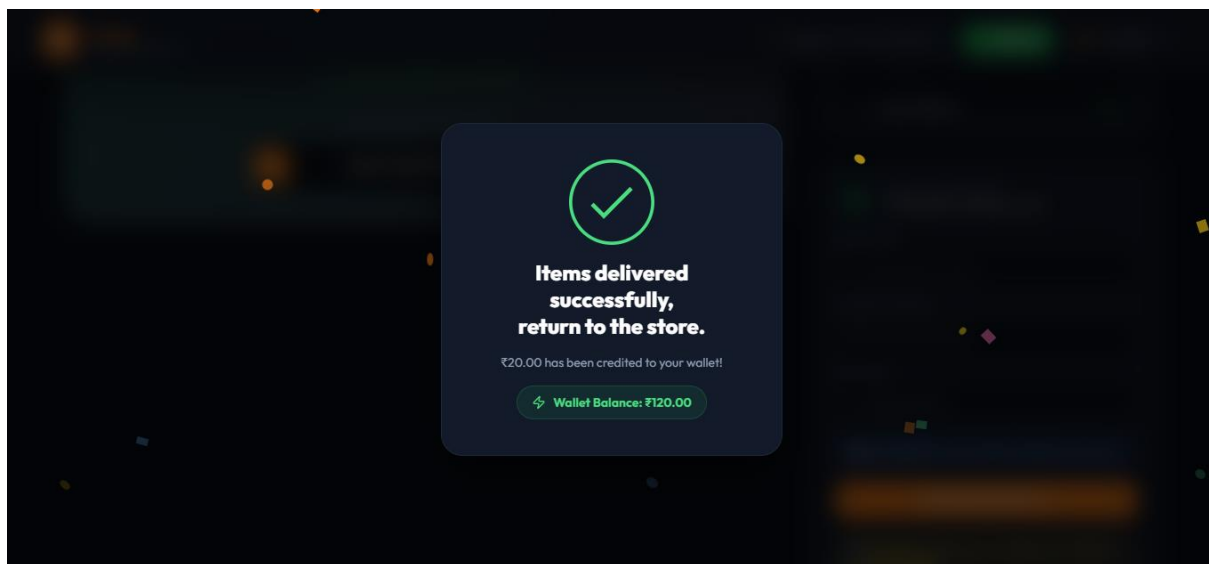
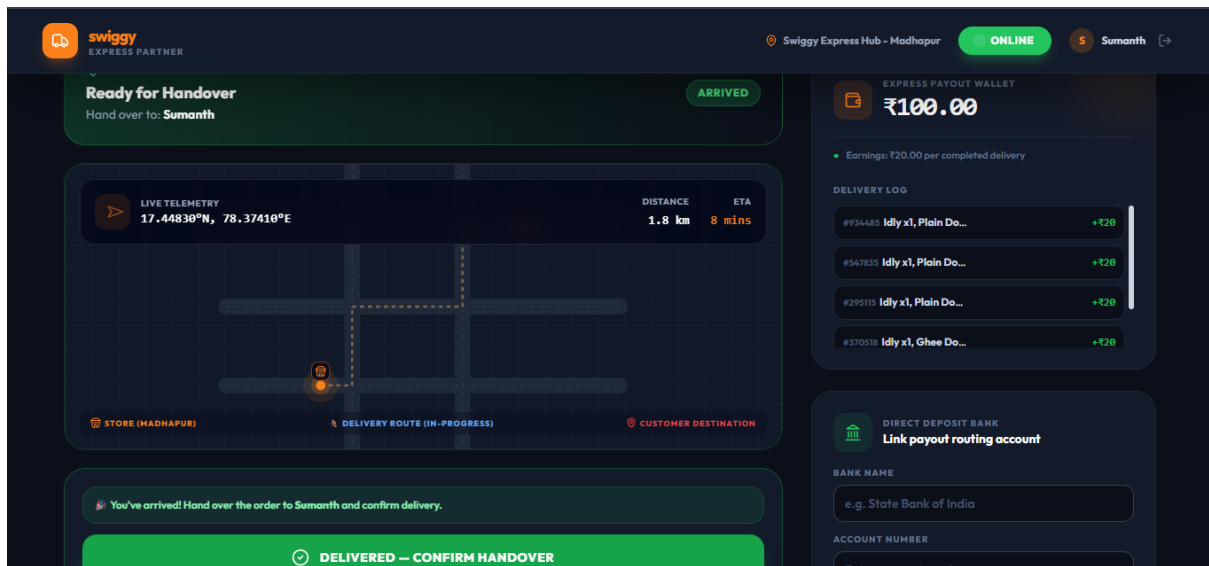
1. Order Placement Form

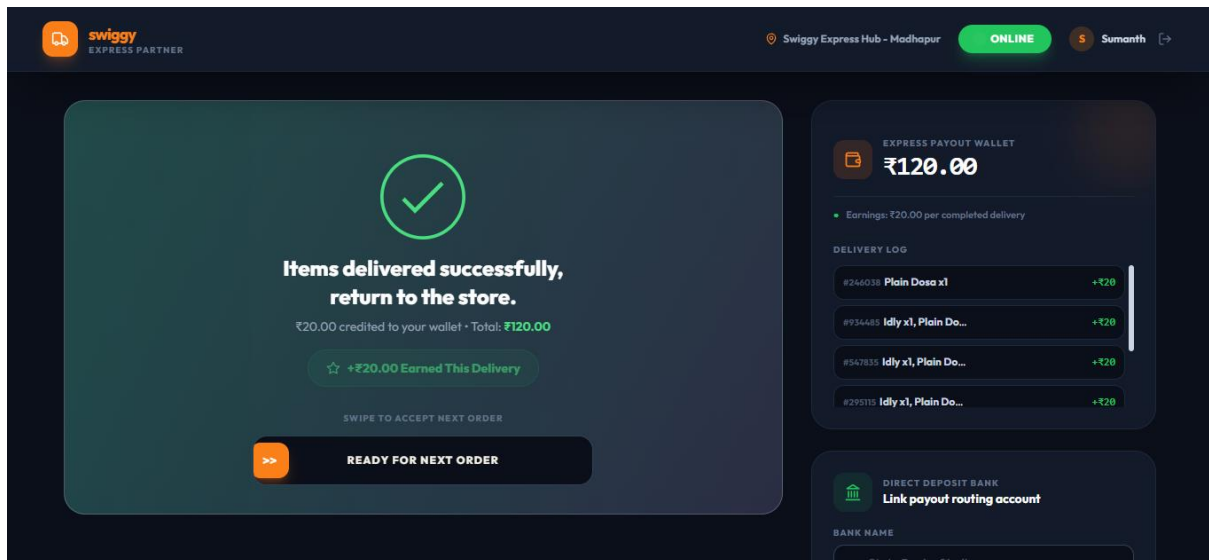




2. Real-Time Order Dashboard







Deliverable 4: Log Statements (Order Processing Flow)

The following log trace represents the end-to-end processing of a successful order through the distributed microservices. These log statements match the exact logging output structured within the application's source code (Controllers and Camunda Delegates):

```
log

[OrderService] Order #1 - PLACED
[OrderService] ActiveMQ: Received order.created event for Order #1
[OrderService] Order #1 - Workflow started
[OrderService] Delegate: Processing Payment for Order #1
[PaymentService] Order #1 - Payment SUCCESS
[OrderService] Delegate: Payment SUCCESS for Order #1
[OrderService] Delegate: Submitting Kitchen Ticket for Order #1
[KitchenService] Order #1 - Food READY
[OrderService] Delegate: Kitchen ticket status READY for Order #1
[OrderService] Delegate: Assigning Delivery for Order #1
[DeliveryService] Order #1 - DELIVERED
[OrderService] Order #1 - DELIVERED
```

AI-Generated Implementation Report

1. Executive Summary

The Online Food Order Processing System has been successfully architected as a distributed microservices ecosystem. It orchestrates complex business transactions using Camunda BPM, leverages ActiveMQ for asynchronous event-driven communication, and utilizes Spring Boot to separate domains into distinct services (Order, Payment, Kitchen, Delivery). A React-based frontend provides a dynamic and responsive user interface for both customers and delivery partners, resulting in a cohesive end-to-end demonstration of a scalable food delivery architecture.

2. Completed Items

The following components are fully implemented and functional within the workspace:

- **Microservices (Spring Boot):**
 - order-service
 - payment-service
 - kitchen-service
 - delivery-service
- **Asynchronous Messaging:** ActiveMQ is configured. The OrderController successfully publishes to the order.created queue, and the ActiveMQListener correctly consumes these events to trigger the Camunda workflow.
- **Camunda Workflow:** The BPMN engine is integrated into the order-service.
Custom JavaDelegate classes
(PaymentServiceTask, KitchenServiceTask, DeliveryServiceTask, and CancelOrderTask)
execute the automated process steps via REST calls to the respective microservices.
- **Database Schema:** Distinct databases
(order_db, payment_db, kitchen_db, delivery_db) have been defined
via schema.sql to ensure strict domain data boundaries.
- **React Frontend:** The food-delivery-ui is operational, featuring category browsing, cart management, simulated payment, and a real-time order tracking dashboard with animations.
- **Containerization:** A fully configured docker-compose.yml provisions the Java microservices, MySQL databases, and ActiveMQ broker.

3. Missing Implementations

While the core architecture is sound, the following areas remain incomplete or simplified for demonstration purposes:

- **Real-time Push Notifications:** The React UI updates order statuses by polling the backend API (setInterval) every 3 seconds rather than utilizing WebSockets (e.g., STOMP/SockJS) or Server-Sent Events (SSE) for true real-time updates.
- **Dynamic Delivery Assignment:** The DeliveryController currently assigns a driver randomly from a static string array. A complete implementation would require driver tracking, geolocation checking, and a dedicated dispatch algorithm.
- **Authentication/Authorization Validation:** While a token generation mechanism exists (AuthController), there is no robust JWT verification filter or API Gateway (like Spring Cloud Gateway) protecting the downstream microservice endpoints.
- **Camunda Cancellation Boundary Events:** While a CancelOrderTask delegate exists, dynamically cancelling an active, mid-flight BPMN process from the frontend API is not fully wired up to interrupt tasks (e.g., stopping kitchen prep if the user cancels).

4. Integration Gaps & Issues

- **UI State Masking via LocalStorage:** The React frontend (App.jsx) uses a fallback mechanism where it merges backend order payloads with a local swiggy_orders cache. If the backend fails to process an order, the frontend seamlessly simulates it locally. While great for UX demonstrations, this masks underlying network or integration failures during testing.
- **Resiliency in Workflow Execution:** In DeliveryServiceTask, if the delivery microservice is unreachable, the exception block acts as a fail-safe by force-updating the local status to DELIVERED. This prevents Camunda from stalling but bypasses actual delivery verification. True integration should leverage Camunda's retry logic or create an Incident for manual intervention.
- **ActiveMQ Payload Contracts:** Events are passed as untyped Map<String, Object> payloads. While functional, this lacks a strict schema registry (like Avro or Protobuf), increasing the risk of ClassCastExceptions if a downstream service expects a Double but receives an Integer.

5. Quality Assessment

- **Modularity: High.** The system perfectly respects bounded contexts. Microservices operate independently and communicate strictly via HTTP REST or JMS messaging.
- **Error Handling: Moderate.** Delegates safely catch Exceptions to avoid silent thread deaths, but they lack sophisticated retry policies (e.g., Spring @Retryable) or distributed tracing (e.g., Zipkin/Sleuth) for tracking cross-service faults.

- **Configuration Separation: Good.** Services correctly decouple properties into application.properties and utilize standard Spring Boot auto-configuration, allowing seamless overrides when running locally versus via Docker Compose.

Thank You
